WNDRVR
SUPPORT NETWORK

VXWORKS 7 TI_TDA4 BSP SUPPLEMENT GUIDE, SR0650 EDITION 3

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 TI_TDA4 BSP Supplement Guide, SR0650 Edition 3

17 August 2020

1. INTRODUCTION

1. Introducing the ti_tda4 BSP

This document describes the features of the ti_tda4 BSP, which is designed to run on the TI J721e EVM development board.

This document must be read in conjunction with the BSP readme file *ti_j721e_evm_readme.md*.

While the BSP readme file guides you through a simple target boot process, this document gives a more complete description of the BSP and its capabilities.

Before using a board with VxWorks®, verify that the board runs in the factory configuration, and check all target board connections. Refer to the board vendor documentation for assistance.

2. Introducing the Boot Process

This BSP supports booting from on board flash memory using the U-Boot boot loader.

The following approaches can be taken when booting the target board. Different file formats and DTB file configurations are available.

Table 1. Target Boot Strategies

Boot Loader	Image File Loaded By Boot Loader	Image File Data Model	Image File Name	Independent or Embedded DTB
U-Boot	VxWorks kernel	32-bit or 64-bit	uVxWorks	Independent and Embedded
U-Boot	VxWorks kernel	32-bit or 64-bit	vxWorks.bin	Embedded
U-Boot	VxWorks boot application	32-bit or 64-bit	vxWorks.bin	Embedded
U-Boot	VxWorks boot application	32-bit or 64-bit	uVxWorks	Embedded

Note: If you follow the process described in Update the BSP DTS File on page 6 and modify the BSP DTS files, then only independent DTB mode can be used. If you do not modify the BSP DTS files then either independent or embedded DTB mode can be used.

3. Configure Development Environment

Initialise the workstation command shell for VxWorks Board Support Package (BSP) configuration and project build.

If your workstation runs Windows, refer to Setting Up a Windows Development Environment on page 2 for instructions.

If your workstation runs Linux, refer to Setting Up a Linux Development Environment on page 2 for instructions.

Setting Up a Windows Development Environment on page 2 The Windows command-line interface must be configured correctly for use with VxWorks.
Setting Up a Linux Development Environment on page 2 The Linux command-line interface must be configured correctly for use with VxWorks.

3.1. Setting Up a Windows Development Environment

The Windows command-line interface must be configured correctly for use with VxWorks.

Before You Begin

Before you begin you must install Wind River VxWorks 7 on your host workstation.

About This Task

These steps configure the workstation command terminal environment variables to allow for VxWorks command-line interface (CLI) development tasks to be performed.

Procedure

1. Change into the Wind River directory.

```
C:\> cd WindRiver
```

2. Add Wind River build tools to the command terminal path.

```
C:\WindRiver> wrenv -p vxworks-7
Microsoft Windows [Version 6.1.7601]
Copyright © 2009 Microsoft Corporation. All rights reserved.

C:\WindRiver>
```

3. Change into the Workbench workspace directory.

```
C:\WindRiver> cd workspace
C:\WindRiver\workspace>
```

Parent topic: Configure Development Environment on page 1

3.2. Setting Up a Linux Development Environment

The Linux command-line interface must be configured correctly for use with VxWorks.

Before You Begin

Before you begin you must install Wind River VxWorks 7 on your host workstation.

About This Task

These steps configure the workstation command terminal environment variables to allow for VxWorks command-line interface (CLI) development tasks to be performed.

Procedure

1. Change into the Wind River directory.

```
~/home/user$ cd WindRiver
```

2. Add Wind River build tools to the command terminal path.

```
~/home/user/WindRiver$ ./wrenv.linux -p vxworks-7
~/home/user/WindRiver$
```

3. Create the workspace directory.

```
~/home/user/WindRiver$ mkdir workspace
~/home/user/WindRiver$
```

4. Change into the workspace directory.

```
~/home/user/WindRiver$ cd workspace
~/home/user/WindRiver/workspace$
```

Parent topic: Configure Development Environment on page 1

2. CONFIGURATION AND SETUP

1. Device Tree and Device Tree Blob Files

This board support package (BSP) uses device tree nodes to describe the device specific information on the board. The supported drivers will get configuration data from the device tree source files (suffixed .dts) that are located in the BSP directory.

For a description of the device tree source file structure, refer to the device tree specification.

The DTS file for this board is called **j721e-evm.dts**.

The device tree source file can be used in either of two ways in your VxWorks projects:

1. Embedded DTB mode is selected when the VSB option **STANDALONE_DTB** is set to **y**. When the VIP is created, the device tree blob is embedded in the VxWorks image. The VIP parameter **DTS_FILE** designates which DTS file in the BSP directory is used by this VIP, and is set according to the table above. When configured this way, by default the VxWorks kernel or boot application image VxWorks boot line is retrieved from the DTS file's chosen node.
2. Independent DTB mode is selected when the VSB option **STANDALONE_DTB** is set to **n**. The corresponding DTB file is produced as part of the VIP build process and stored in the default sub-directory in the BSP directory. These files are loaded from the U-Boot command shell using the **bootm** command..

2. Program U-Boot onto the Target Board

Refer to *ti_j721e_evm_readme.md* for information on how to program U-Boot on to the target board.

3. VXWORKS PROJECTS

1. Project Creation Workflow

A VxWorks source build (VSB) project must always be created before the VxWorks image project (VIP).

When creating a VSB using the ti_tda4 BSP, Wind River recommends you also specify the name of the Board Support Package (BSP) at VSB creation time, as this automatically configures the VSB with the necessary VxWorks layers.

When creating the VSB, you have the option of specifying the CPU instead of the BSP. If you do this, you must remember to add the correct VxWorks layers to the VSB before building the project. Refer to Creating a 32-bit Symmetric Multiprocessor VxWorks Source Build on page 18 for details on how to create the VSB project and add VxWorks layers to it.

From the command-line, you specify the BSP in the VSB creation command by adding -bsp ti_tda4. You specify the CPU in the VSB creation command as follows: 32-bit CPU in the VSB creation command by adding -cpu ARMARCH7 to the VSB creation command.

Table 1. CPU Settings for ti_tda4 BSP

Data Size	VSB Creation Command Option
32-bit	-cpu ARMARCH7
64-bit	-cpu ARMARCH8

Note: The project creation workflow can also be achieved within the Workbench Eclipse IDE. For details, refer to *Workbench 4 Getting Started: Workbench Project Overview*

2. Processing Environments and Data Models

When building projects with this Board Support Package (BSP) you can choose between two processing environments and data models.

The following variations are supported by the BSP:

- ARM v7 - 32-bit arm CPU symmetric multiprocessor
- ARM v8 - 64-bit arm CPU symmetric multiprocessor

Note: All 32-bit arm projects are also referred to as using ARM v7, or ARMARCH7 build spec. The 64-bit projects are ARM v8, or ARMARCH8A.

Table 1. Processing Environment and Data Model VSB Options

Data Model Option	Parameter Specified During VSB Creation
ARM v8 64-bit Data Width	-lp64 (default)
ARM v7 32-bit Data Width	-ilp32

Data Model Option	Parameter Specified During VSB Creation
Symmetric Multiprocessor	-smp (default)

Note: When creating a VxWorks Source Build (VSB), if you do not specify these options on creation, the project defaults to 64-bit symmetric multiprocessor.

3. Update the BSP DTS File

Before using the remote processor or mailbox BSP features, you must modify the Wind River supplied BSP DTS files.

The BSP DTS files must be modified before you create your VxWorks projects. Locate the affected DTS nodes in the vendor supplied Linux DTS files **j721e-som-p0.dtsi**, **k3-j721e-main.dtsi**, and **k3-j721e-common-proc-board.dts**. These nodes must be migrated to the Wind River supplied BSP DTS files **j721e-evm.dts**, and **tda4.dtsi**. Once the copy is complete, further modifications must be made to the BSP DTS file nodes.

Note: Once you modify the Wind River supplied DTS files, you must not attempt to use the BSP in embedded DTB mode. Only independent DTB mode is supported once manual changes are applied to the Wind River DTS files.

Locating the Vendor DTS Files on page 6 Locate the vendor supplied DTS files and copy them to a USB drive.
Copying the DTS Nodes Between Vendor and Wind River Supplied DTS Files on page 7 Manually copy nodes from the vendor DTS files to the Wind River BSP DTS files.
Modifying the BSP DTS File on page 15 Manually modify settings in the Wind River DTS files.

3.1. Locating the Vendor DTS Files

Locate the vendor supplied DTS files and copy them to a USB drive.

Before You Begin

You must use a Linux workstation to retrieve the vendor supplied Device Tree Source (DTS) files.

These steps assist you in locating three vendor supplied DTS files:

- **k3-j721e-som-p0.dtsi**
- **k3-j721e-main.dtsi**
- **k3-j721e-common-proc-board.dts**

Once the files are located, you must copy them to a USB drive that is attached to the Linux workstation and mounted as the device **/dev/usb0/**.

Procedure

1. Download the RTOS SDK tar file PROCESSOR-SDK-RTOS-AUTOMOTIVE06_02_00_21.

   Locate the tar file at **https://software-dl.ti.com/jacinto7/esd/processor-sdk-rtos-jacinto7/06_02_00_21/exports/psdk_rtos_auto_j7_06_02_00_21.tar.gz**.

2. Extract the tar file.

```
linux-host:~% tar -xvf psdk_rtos_auto_j7_06_02_00_21.tar.gz
```

3. Locate the vendor supplied DTS file **j721e-som-p0.dtsi**.

```
linux-host:~% cd psdk_rtos_auto_j7_06_02_00_21/vision_apps/apps/basic_demos/app_linux_fs_files/
dtbo
```

4. Copy **k3-j721e-som-p0.dtsi** to the USB drive.

```
linux-host:~/psdk_rtos_auto_j7_06_02_00_21/vision_apps/apps/basic_demos/app_linux_fs_files/dtbo
% cp k3-j721e-som-p0.dtsi /dev/usb0/.
linux-host:~/psdk_rtos_auto_j7_06_02_00_21/vision_apps/apps/basic_demos/app_linux_fs_files/dtbo
% cd ~/
```

5. Download the Linux SDK installer PROCESSOR-SDK-LINUX-AUTOMOTIVE06_02_00_07.

   Locate the installer at **https://software-dl.ti.com/jacinto7/esd/processor-sdk-linux-jacinto7/06_02_00_07/exports/ti-processor-sdk-linux-automotive-j7-evm-06_02_00-Linux-x86-Install.bin**.

6. Make sure the installer file is an executable Linux file.

```
linux-host:~% chmod a+x ti-processor-sdk-linux-automotive-j7-evm-06_02_00-Linux-x86-Install.bin
```

7. Run the Linux SDK installer.

```
linux-host:~% ./ti-processor-sdk-linux-automotive-j7-evm-06_02_00-Linux-x86-Install.bin
```

8. Locate the vendor supplied DTS files **k3-j721e-main.dtsi** and **k3-j721e-common-proc-board.dts**.

```
linux-host:~% cd ti-processor-sdk-linux-automotive-j7-evm-06_02_00/board-support/linux-4.19.94+
gitAUTOINC+5a23bc00e0-g5a23bc00e0/arch/arm64/boot/dts/ti
```

9. Copy **k3-j721e-main.dtsi** and **k3-j721e-common-proc-board.dts** to the USB drive.

```
linux-host:~ti-processor-sdk-linux-automotive-j7-evm-06_02_00/board-support/linux-4.19.94+gitAU
TOINC+5a23bc00e0-g5a23bc00e0/arch/arm64/boot/dts/ti% cp k3-j721e-main.dtsi /dev/usb0/.
linux-host:~ti-processor-sdk-linux-automotive-j7-evm-06_02_00/board-support/linux-4.19.94+gitAU
TOINC+5a23bc00e0-g5a23bc00e0/arch/arm64/boot/dts/ti% cp k3-j721e-common-proc-board.dts /dev/usb
0/.
```

Parent topic: Update the BSP DTS File on page 6

3.2. Copying the DTS Nodes Between Vendor and Wind River Supplied DTS Files

Manually copy nodes from the vendor DTS files to the Wind River BSP DTS files.

Before You Begin

These steps require that you have completed the steps given in Locating the Vendor DTS Files on page 6.

About This Task

These steps assume you have attached a USB drive containing the vendor supplied DTS files to the workstation:

- **k3-j721e-som-p0.dtsi**
- **k3-j721e-main.dtsi**
- **k3-j721e-common-proc-board.dts**

Procedure

1. Open **k3-j721e-som-p0.dtsi** in an editor.
2. Locate the Wind River supplied file **j721e-evm.dts** in the BSP directory and open the file in an editor.
3. Locate the node **reserved_memory** in **k3-j721e-som-p0.dtsi**.
4. Locate the node **memory@80000000** in **j721e-evm.dts**.
5. Copy the **reserved_memory** node with the following specifiers and insert this immediately following the node **memory@80000000** in **j721e-evm.dts**.:
   - **main_r5fss0_core0_dma_memory_region**
   - **main_r5fss0_core0_memory_region**

```
    memory@80000000
        {
        device_type = "memory";
        reg = <0x0 0x80000000 0x0 0x80000000>,
              <0x8 0x80000000 0x0 0x80000000>;
        };

        reserved_memory: reserved-memory
        {
        #address-cells = <2>;
        #size-cells = <2>;
        ranges;

        main_r5fss0_core0_dma_memory_region: r5f-dma-memory@a2000000
            {
            compatible = "shared-dma-pool";
            reg = <0x00 0xa2000000 0x00 0x100000>;
            no-map;
            };

        main_r5fss0_core0_memory_region: r5f-memory@a2100000
            {
            compatible = "shared-dma-pool";
            reg = <0x00 0xa2100000 0x00 0xf00000>;
            no-map;
            };
        };
```

6. Locate the node **&main_r5fss0_core0** in **k3-j721e-som-p0.dtsi**.
7. Copy the **&main_r5fss0_core0** node and append to the end of **j721e-evm.dts**:

```
&main_r5fss0_core0
    {
    memory-region = <&main_r5fss0_core0_dma_memory_region>,
                    <&main_r5fss0_core0_memory_region>;
    };
```

8. Open **k3-j721e-main.dtsi** in an editor.
9. Locate the Wind River supplied file **tda4.dtsi** in the BSP directory and open in an editor.
10. Locate the node **mailbox0_cluster1** in **k3-j721e-main.dtsi**.

11. Copy the **mailbox0_cluster1** node and append it to the end of the **soc** node in **tda4.dtsi**.

```
        dummy_cmn_refclk1: dummy_cmn_refclk1
            {
            #clock-cells = <0>;
            compatible = "fixed-clock";
            };

        mailbox0_cluster1: mailbox@31f81000
            {
            compatible = "ti,am654-mailbox";
            reg = <0x00 0x31f81000 0x00 0x200>;
            #mbox-cells = <1>;
            ti,mbox-num-users = <4>;
            ti,mbox-num-fifos = <16>;
            interrupt-parent = <&main_navss_intr>;
            interrupts = <215 0 IRQ_TYPE_LEVEL_HIGH>;

            mbox_main_r5fss0_core0: mbox-main-r5fss0-core0
                 {
                 ti,mbox-rx = <0 0 0>;
                 ti,mbox-tx = <1 0 0>;
                 };

            mbox_main_r5fss0_core1: mbox-main-r5fss0-core1
                 {
                 ti,mbox-rx = <2 0 0>;
                 ti,mbox-tx = <3 0 0>;
                 };
                     };
        }; /* SOC */
    };/* end of root */
```

12. Locate the node **main_r5fss0** in **k3-j721e-main.dtsi**.
13. Copy the **main_r5fss0** node and append it to the end of the **soc** node in **tda4.dtsi**.

```
            mbox_main_r5fss0_core0: mbox-main-r5fss0-core0
                 {
                 ti,mbox-rx = <0 0 0>;
                 ti,mbox-tx = <1 0 0>;
                 };

            mbox_main_r5fss0_core1: mbox-main-r5fss0-core1
                 {
                 ti,mbox-rx = <2 0 0>;
                 ti,mbox-tx = <3 0 0>;
                 };
                     };

            main_r5fss0: r5fss@5c00000
                 {
                 compatible = "ti,j721e-r5fss";
                 lockstep-mode = <0>;
                 #address-cells = <1>;
                 #size-cells = <1>;
                 ranges = <0x5c00000 0x00 0x5c00000 0x20000>,
                          <0x5d00000 0x00 0x5d00000 0x20000>;
                 power-domains = <&k3_pds 243 1>;
```

```
                main_r5fss0_core0: r5f@5c00000
                     {
                     compatible = "ti,j721e-r5f";
                     reg = <0x5c00000 0x00008000>,
                           <0x5c10000 0x00008000>;
                     reg-names = "atcm", "btcm";
                     ti,sci = <&dmsc>;
                     ti,sci-dev-id = <245>;
                     ti,sci-proc-ids = <0x06 0xFF>;
                     resets = <&k3_reset 245 1>;
                     atcm-enable = <1>;
                     btcm-enable = <1>;
                     loczrama = <1>;
                     mboxes = <&mailbox0_cluster1 &mbox_main_r5fss0_core0>;
                     };

                main_r5fss0_core1: r5f@5d00000
                     {
                     compatible = "ti,j721e-r5f";
                     reg = <0x5d00000 0x00008000>,
                           <0x5d10000 0x00008000>;
                     reg-names = "atcm", "btcm";
                     ti,sci = <&dmsc>;
                     ti,sci-dev-id = <246>;
                     ti,sci-proc-ids = <0x07 0xFF>;
                     resets = <&k3_reset 246 1>;
                     atcm-enable = <1>;
                     btcm-enable = <1>;
                     loczrama = <1>;
                     mboxes = <&mailbox0_cluster1 &mbox_main_r5fss0_core1>;
                     };
                };
            }; /* SOC */
        };/* end of root */
```

14. Locate the twelve nodes **mailbox0_cluster0** to **mailbox0_cluster11** in **k3-j721e-main.dtsi**.
15. Copy the twelve nodes **mailbox0_cluster0** to **mailbox0_cluster11** and append them to the end of node **soc** in **tda4.dtsi**.

```
                main_r5fss0_core1: r5f@5d00000
                     {
                     compatible = "ti,j721e-r5f";
                     reg = <0x5d00000 0x00008000>,
                           <0x5d10000 0x00008000>;
                     reg-names = "atcm", "btcm";
                     ti,sci = <&dmsc>;
                     ti,sci-dev-id = <246>;
                     ti,sci-proc-ids = <0x07 0xFF>;
                     resets = <&k3_reset 246 1>;
                     atcm-enable = <1>;
                     btcm-enable = <1>;
                     loczrama = <1>;
                     mboxes = <&mailbox0_cluster1 &mbox_main_r5fss0_core1>;
                     };
                };

                mailbox0_cluster0: mailbox@31f80000
```

```
                    {
                    compatible = "ti,am654-mailbox";
                    reg = <0x00 0x31f80000 0x00 0x200>;
                    #mbox-cells = <1>;
                    ti,mbox-num-users = <4>;
                    ti,mbox-num-fifos = <16>;
                    interrupt-parent = <&main_navss_intr>;
                    interrupts = <214 0 IRQ_TYPE_LEVEL_HIGH>;

                    mbox_mcu_r5fss0_core0: mbox-mcu-r5fss0-core0
                        {
                                        ti,mbox-rx = <0 0 0>;
                                        ti,mbox-tx = <1 0 0>;
                                    };

                       mbox_mcu_r5fss0_core1: mbox-mcu-r5fss0-core1
                        {
                                        ti,mbox-rx = <2 0 0>;
                                        ti,mbox-tx = <3 0 0>;
                                    };
                         };

                mailbox0_cluster1: mailbox@31f81000
                    {
                    compatible = "ti,am654-mailbox";
                    reg = <0x00 0x31f81000 0x00 0x200>;
                    #mbox-cells = <1>;
                    ti,mbox-num-users = <4>;
                    ti,mbox-num-fifos = <16>;
                    interrupt-parent = <&main_navss_intr>;
                    interrupts = <215 0 IRQ_TYPE_LEVEL_HIGH>;

                    mbox_main_r5fss0_core0: mbox-main-r5fss0-core0
                        {
                        ti,mbox-rx = <0 0 0>;
                        ti,mbox-tx = <1 0 0>;
                        };

                    mbox_main_r5fss0_core1: mbox-main-r5fss0-core1
                        {
                        ti,mbox-rx = <2 0 0>;
                        ti,mbox-tx = <3 0 0>;
                        };
                    };

                mailbox0_cluster2: mailbox@31f82000
                    {
                    compatible = "ti,am654-mailbox";
                    reg = <0x00 0x31f82000 0x00 0x200>;
                    #mbox-cells = <1>;
                    ti,mbox-num-users = <4>;
                    ti,mbox-num-fifos = <16>;
                    interrupt-parent = <&main_navss_intr>;
                    interrupts = <216 0 IRQ_TYPE_LEVEL_HIGH>;

                    mbox_main_r5fss1_core0: mbox-main-r5fss1-core0
                        {
```

```
                ti,mbox-rx = <0 0 0>;
                ti,mbox-tx = <1 0 0>;
                };

            mbox_main_r5fss1_core1: mbox-main-r5fss1-core1
                {
                ti,mbox-rx = <2 0 0>;
                ti,mbox-tx = <3 0 0>;
                };
            };

        mailbox0_cluster3: mailbox@31f83000
            {
            compatible = "ti,am654-mailbox";
            reg = <0x00 0x31f83000 0x00 0x200>;
            #mbox-cells = <1>;
            ti,mbox-num-users = <4>;
            ti,mbox-num-fifos = <16>;
            interrupt-parent = <&main_navss_intr>;
            interrupts = <217 0 IRQ_TYPE_LEVEL_HIGH>;

            mbox_c66_0: mbox-c66-0
                {
                ti,mbox-rx = <0 0 0>;
                ti,mbox-tx = <1 0 0>;
                };

            mbox_c66_1: mbox-c66-1
                {
                ti,mbox-rx = <2 0 0>;
                ti,mbox-tx = <3 0 0>;
                };
            };

        mailbox0_cluster4: mailbox@31f84000
            {
            compatible = "ti,am654-mailbox";
            reg = <0x00 0x31f84000 0x00 0x200>;
            #mbox-cells = <1>;
            ti,mbox-num-users = <4>;
            ti,mbox-num-fifos = <16>;
            interrupt-parent = <&main_navss_intr>;
            interrupts = <218 0 IRQ_TYPE_LEVEL_HIGH>;

            mbox_c71_0: mbox-c71-0
                {
                ti,mbox-rx = <0 0 0>;
                ti,mbox-tx = <1 0 0>;
                };
            };

        mailbox0_cluster5: mailbox@31f85000
            {
            compatible = "ti,am654-mailbox";
            reg = <0x00 0x31f85000 0x00 0x200>;
            #mbox-cells = <1>;
            ti,mbox-num-users = <4>;
```

```
                ti,mbox-num-fifos = <16>;
                status = "disabled";
                };

            mailbox0_cluster6: mailbox@31f86000
                {
                compatible = "ti,am654-mailbox";
                reg = <0x00 0x31f86000 0x00 0x200>;
                #mbox-cells = <1>;
                ti,mbox-num-users = <4>;
                ti,mbox-num-fifos = <16>;
                status = "disabled";
                };

            mailbox0_cluster7: mailbox@31f87000
                {
                compatible = "ti,am654-mailbox";
                reg = <0x00 0x31f87000 0x00 0x200>;
                #mbox-cells = <1>;
                ti,mbox-num-users = <4>;
                ti,mbox-num-fifos = <16>;
                status = "disabled";
                };

            mailbox0_cluster8: mailbox@31f88000
                {
                compatible = "ti,am654-mailbox";
                reg = <0x00 0x31f88000 0x00 0x200>;
                #mbox-cells = <1>;
                ti,mbox-num-users = <4>;
                ti,mbox-num-fifos = <16>;
                status = "disabled";
                };

            mailbox0_cluster9: mailbox@31f89000
                {
                compatible = "ti,am654-mailbox";
                reg = <0x00 0x31f89000 0x00 0x200>;
                #mbox-cells = <1>;
                ti,mbox-num-users = <4>;
                ti,mbox-num-fifos = <16>;
                status = "disabled";
                };

            mailbox0_cluster10: mailbox@31f8a000
                {
                compatible = "ti,am654-mailbox";
                reg = <0x00 0x31f8a000 0x00 0x200>;
                #mbox-cells = <1>;
                ti,mbox-num-users = <4>;
                ti,mbox-num-fifos = <16>;
                status = "disabled";
                };

            mailbox0_cluster11: mailbox@31f8b000
                {
                compatible = "ti,am654-mailbox";
```

```
                    reg = <0x00 0x31f8b000 0x00 0x200>;
                    #mbox-cells = <1>;
                    ti,mbox-num-users = <4>;
                    ti,mbox-num-fifos = <16>;
                    status = "disabled";
                    };
        }; /* SOC */
    };/* end of root */
```

16. Open **k3-j721e-common-proc-board.dts** in an editor.
17. Locate the node **cpsw9g_virt_mac** in **k3-j721e-common-proc-board.dts**.
18. Copy the node **cpsw9g_virt_mac** and append it to the end of the **soc** node in **tda4.dtsi**.

```
        cpsw9g_virt_mac: main_r5fss_cpsw9g_virt_mac0
            {
            compatible = "ti,j721e-cpsw-virt-mac";
            dma-coherent;
            ti,psil-base = <0x4a00>;
            ti,remote-name = "mpu_1_0_ethswitch-device-0";
            interrupt-parent = <&main_udmass_inta>;

            dmas = <&main_udmap &cpsw9g_virt_mac 0 UDMA_DIR_TX>,
                   <&main_udmap &cpsw9g_virt_mac 1 UDMA_DIR_TX>,
                   <&main_udmap &cpsw9g_virt_mac 2 UDMA_DIR_TX>,
                   <&main_udmap &cpsw9g_virt_mac 3 UDMA_DIR_TX>,
                   <&main_udmap &cpsw9g_virt_mac 4 UDMA_DIR_TX>,
                   <&main_udmap &cpsw9g_virt_mac 5 UDMA_DIR_TX>,
                   <&main_udmap &cpsw9g_virt_mac 6 UDMA_DIR_TX>,
                   <&main_udmap &cpsw9g_virt_mac 0 UDMA_DIR_RX>;
            dma-names = "tx0", "tx1", "tx2", "tx3",
                        "tx4", "tx5", "tx6",
                        "rx";

            virt_emac_port
                {
                ti,label = "virt-port";
                /* local-mac-address = [0 0 0 0 0 0]; */
                };

            ti,psil-config0
                {
                linux,udma-mode = <UDMA_PKT_MODE>;
                statictr-type = <PSIL_STATIC_TR_NONE>;
                ti,needs-epib;
                ti,psd-size = <16>;
                };

            ti,psil-config1
                {
                linux,udma-mode = <UDMA_PKT_MODE>;
                statictr-type = <PSIL_STATIC_TR_NONE>;
                ti,needs-epib;
                ti,psd-size = <16>;
                };

            ti,psil-config2
```

```
                {
                linux,udma-mode = <UDMA_PKT_MODE>;
                statictr-type = <PSIL_STATIC_TR_NONE>;
                ti,needs-epib;
                ti,psd-size = <16>;
                };

            ti,psil-config3
                {
                linux,udma-mode = <UDMA_PKT_MODE>;
                statictr-type = <PSIL_STATIC_TR_NONE>;
                ti,needs-epib;
                ti,psd-size = <16>;
                };

            ti,psil-config4
                {
                linux,udma-mode = <UDMA_PKT_MODE>;
                statictr-type = <PSIL_STATIC_TR_NONE>;
                ti,needs-epib;
                ti,psd-size = <16>;
                };

            ti,psil-config5
                {
                linux,udma-mode = <UDMA_PKT_MODE>;
                statictr-type = <PSIL_STATIC_TR_NONE>;
                ti,needs-epib;
                ti,psd-size = <16>;
                };

            ti,psil-config6
                {
                linux,udma-mode = <UDMA_PKT_MODE>;
                statictr-type = <PSIL_STATIC_TR_NONE>;
                ti,needs-epib;
                ti,psd-size = <16>;
                };

            ti,psil-config7
                {
                linux,udma-mode = <UDMA_PKT_MODE>;
                statictr-type = <PSIL_STATIC_TR_NONE>;
                ti,needs-epib;
                ti,psd-size = <16>;
                };
            };
        }; /* SOC */
    };/* end of root */
```

Parent topic: Update the BSP DTS File on page 6

3.3. Modifying the BSP DTS File

Manually modify settings in the Wind River DTS files.

Before You Begin

These steps assume you have completed the steps outlined in Copying the DTS Nodes Between Vendor and Wind River Supplied DTS Files on page 7

About This Task

These steps explain how to make necessary modifications to BSP DTS files **tda4.dtsi** and **j721e-evm.dts**.

Procedure

1. Open **tda4.dtsi** in an editor.
2. Locate the node **cpsw9g_virt_mac**.
3. Locate **interrupt-parent** and change to **msi-parent**.

```
        cpsw9g_virt_mac: main_r5fss_cpsw9g_virt_mac0 {
                compatible = "ti,j721e-cpsw-virt-mac";
                dma-coherent;
                ti,psil-base = <0x4a00>;
                ti,remote-name = "mpu_1_0_ethswitch-device-0";
                msi-parent = <&main_udmass_inta>;

                dmas = <&main_udmap &cpsw9g_virt_mac 0 UDMA_DIR_TX>,
```

4. Locate eight instances of the string **main_udmap** and change to **main_udma**.

```
                msi-parent = <&main_udmass_inta>;

                dmas = <&main_udma &cpsw9g_virt_mac 0 UDMA_DIR_TX>,
                       <&main_udma &cpsw9g_virt_mac 1 UDMA_DIR_TX>,
                       <&main_udma &cpsw9g_virt_mac 2 UDMA_DIR_TX>,
                       <&main_udma &cpsw9g_virt_mac 3 UDMA_DIR_TX>,
                       <&main_udma &cpsw9g_virt_mac 4 UDMA_DIR_TX>,
                       <&main_udma &cpsw9g_virt_mac 5 UDMA_DIR_TX>,
                       <&main_udma &cpsw9g_virt_mac 6 UDMA_DIR_TX>,
                       <&main_udma &cpsw9g_virt_mac 0 UDMA_DIR_RX>;
```

5. Locate eight instances of the string **linux,udma-mode** and change to **udma-mode**.

```
                ti,psil-config0 {
                        udma-mode = <UDMA_PKT_MODE>;
                        statictr-type = <PSIL_STATIC_TR_NONE>;
                        ti,needs-epib;
                        ti,psd-size = <16>;
                };

                ti,psil-config1 {
                        udma-mode = <UDMA_PKT_MODE>;
                        statictr-type = <PSIL_STATIC_TR_NONE>;
                        ti,needs-epib;
                        ti,psd-size = <16>;
                };
```

```
                ti,psil-config2 {
                        udma-mode = <UDMA_PKT_MODE>;
                        statictr-type = <PSIL_STATIC_TR_NONE>;
                        ti,needs-epib;
                        ti,psd-size = <16>;
                };

                ti,psil-config3 {
                        udma-mode = <UDMA_PKT_MODE>;
                        statictr-type = <PSIL_STATIC_TR_NONE>;
                        ti,needs-epib;
                        ti,psd-size = <16>;
                };

                ti,psil-config4 {
                        udma-mode = <UDMA_PKT_MODE>;
                        statictr-type = <PSIL_STATIC_TR_NONE>;
                        ti,needs-epib;
                        ti,psd-size = <16>;
                };

                ti,psil-config5 {
                        udma-mode = <UDMA_PKT_MODE>;
                        statictr-type = <PSIL_STATIC_TR_NONE>;
                        ti,needs-epib;
                        ti,psd-size = <16>;
                };

                ti,psil-config6 {
                        udma-mode = <UDMA_PKT_MODE>;
                        statictr-type = <PSIL_STATIC_TR_NONE>;
                        ti,needs-epib;
                        ti,psd-size = <16>;
                };

                ti,psil-config7 {
                        udma-mode = <UDMA_PKT_MODE>;
                        statictr-type = <PSIL_STATIC_TR_NONE>;
                        ti,needs-epib;
                        ti,psd-size = <16>;
                };
        };
```

6. Locate the line containing **virt_emac_port**.

```
                virt_emac_port {
                        ti,label = "virt-port";
                        /* local-mac-address = [0 0 0 0 0 0]; */
                };
```

7. Add a line containing **status = "disabled"** above this line.

```
        status = "disabled";

                virt_emac_port {
                        ti,label = "virt-port";
```

```
                        /* local-mac-address = [0 0 0 0 0 0]; */
                };
```

8. Open the file **j721e-evm.dts** in an editor.
9. Append the following text to the end of **j721e-evm.dts**:

```
                mcu_r5fss0_core0_memory_region: r5f-memory@a0100000 {
                        compatible = "shared-dma-pool";
                        reg = <0x00 0xa0100000 0x00 0xf00000>;
                        no-map;
                };
        };

&main_r5fss0_core0 {
        memory-region = <&main_r5fss0_core0_dma_memory_region>,
                        <&main_r5fss0_core0_memory_region>;
};

&cpsw9g_virt_mac
  {
  status = "okay";
  };
```

10. Do a search for every instance of the string **IRQ_TYPE_LEVEL_HIGH** in BSP DTS files **tda4.dtsi** and **j721e-evm.dts**.
    1. Delete each instance of **IRQ_TYPE_LEVEL_HIGH**.
    2. In place of **IRQ_TYPE_LEVEL_HIGH** insert the number 4.
11. Do a search for every instance of the string **TI_SCI_PD_EXCLUSIVE** in BSP DTS files **tda4.dtsi** and **j721e-evm.dts**.
    1. Delete each instance of **TI_SCI_PD_EXCLUSIVE**.
    2. In place of **TI_SCI_PD_EXCLUSIVE** insert the number 1.

Parent topic: Update the BSP DTS File on page 6

4. Creating a 32-bit Symmetric Multiprocessor VxWorks Source Build

Before You Begin

Ensure your VxWorks development environment is correctly configured. Refer to Configure Development Environment on page 1.

Procedure

1. Create the VxWorks source build (VSB).

```
C:\WindRiver\workspace> vxprj vsb create -force -ilp32 -bsp ti_tda4 -S myVSB
C:\WindRiver\workspace> cd myVSB
```

Note: Alternatively, you can base your 32-bit VSB on a CPU rather than a BSP:

```
C:\WindRiver\workspace> vxprj vsb create -force -ilp32 -cpu ARMARCH7 -S myVSB
C:\WindRiver\workspace> cd myVSB
```

2. **Optional:** Add support for the Large Physical Address Extension architecture (LPAE):

```
C:\WindRiver\workspace\myVSB> vxprj vsb config -s -add _WRS_CONFIG_ARM_LPAE=y
```

3. Build the VSB.

```
C:\WindRiver\workspace\myVSB> vxprj build
```

5. Creating a Boot Application Project

Before You Begin

Make sure you have already build a VSB project. For instructions on building a 64-bit VSB, refer to *ti_j721e_evm_readme.md*. For a 32-bit VSB, refer to Creating a 32-bit Symmetric Multiprocessor VxWorks Source Build on page 18.

About This Task

In these steps you will build a VxWorks boot application image.

Procedure

1. Create a VxWorks image project (VIP) with the bootapp profile selected.

```
C:\WindRiver\workspace> vxprj vip create -vsb myVSB -profile PROFILE_BOOTAPP ti_tda4 myVIP
```

2. Build the boot application VIP.

```
C:\WindRiver\workspace\myBootApp> vxprj build vxWorks.bin
```

6. The VxWorks Kernel Image

This BSP supports booting VxWorks as both uVxWorks and vxWorks binary kernel image files. For instructions on building the **uVxWorks** kernel image, refer to *ti_j721e_evm_readme.md*.

Creating a vxWorks.bin Kernel Image on page 20
Creating a VxWorks Kernel Image with Embedded DTB on page 20
Determine the VIP Builds and Build Rules on page 21 Once the VIP is created, determine all its supported build targets from the command line interface.

6.1. Creating a vxWorks.bin Kernel Image

Before You Begin

Make sure you have already build a VSB project. For instructions on building a 64-bit VSB, refer to *ti_j721e_evm_readme.md*. For a 32-bit VSB, refer to Creating a 32-bit Symmetric Multiprocessor VxWorks Source Build on page 18.

About This Task

In these steps you will build a VxWorks kernel binary image.

Procedure

1. Create a VxWorks image project (VIP).

```
C:\WindRiver\workspace> vxprj vip create -vsb myVSB ti_tda4 myVIP
C:\WindRiver\workspace> cd myVIP
```

2. Select an embedded Device Tree Blob (DTB) file.

```
C:\WindRiver\workspace\myVIP> vxprj vip component add myVIP INCLUDE_STANDALONE_DTB
```

3. Build the boot application VIP.

```
C:\WindRiver\workspace\myVIP> vxprj build vxWorks.bin
```

Parent topic: The VxWorks Kernel Image on page 19

6.2. Creating a VxWorks Kernel Image with Embedded DTB

Before You Begin

Make sure you have already build a VSB project. For instructions on building a 64-bit VSB, refer to *ti_j721e_evm_readme.md*. For a 32-bit VSB, refer to Creating a 32-bit Symmetric Multiprocessor VxWorks Source Build on page 18.

About This Task

In these steps you will build a VxWorks kernel binary image.

Procedure

1. Create a VxWorks image project (VIP).

```
C:\WindRiver\workspace> vxprj vip create -vsb myVSB ti_tda4 myVIP
C:\WindRiver\workspace> cd myVIP
```

2. Select an embedded Device Tree Blob (DTB) file.

```
C:\WindRiver\workspace\myVIP> vxprj vip component add myVIP INCLUDE_STANDALONE_DTB
```


WNDRVR

3. Build the boot application VIP.

```
C:\WindRiver\workspace\myVIP> vxprj build uVxWorks
```

Parent topic: The VxWorks Kernel Image on page 19

6.3. Determine the VIP Builds and Build Rules

Once the VIP is created, determine all its supported build targets from the command line interface.

Determine the Supported Builds

Usually, VxWorks BSPs allow the VIP to build RAM resident VxWorks image files. However, some BSPs also allow ROM resident VxWorks image files to be built also.

Determine which builds are supported by examining the configuration of the VIP with the command vxprj build list.

For example, lets assume you have created two VIPs using different BSPs and you named them **myVIP** and **my2ndVIP**. You determine the supported builds as follows:

```
C:\WindRiver\workspace> vxprj build list myVIP
default
C:\WindRiver\workspace> vxprj build list my2ndVIP
default
default_romResident
```

The **myVIP** project only supports the **default** build, and so this means you can only build RAM resident VxWorks image files to run on the target board.

The **my2ndVIP** project supports the **default** build, but it also supports the **default_romResident** build. This means you can build both RAM and ROM resident VxWorks image files. If your BSP supports these builds, you can switch a VIP from **default** to **default_romResident** build with the following command:

```
C:\WindRiver\workspace> cd my2ndVIP
C:\WindRiver\workspace\my2ndVIP> vxprj build set default_romResident
```

Building the ROM resident VxWorks image is then achieved as follows:

```
C:\WindRiver\workspace\my2ndVIP> vxprj build vxWorks_romResident
```

Switch back and build the default build as follows:

```
C:\WindRiver\workspace\my2ndVIP> vxprj build set default
C:\WindRiver\workspace\my2ndVIP> vxprj build vxWorks
```

Determine the Supported Buildrules

Buildrules determine which VxWorks image file types you can build from the VIP. Common VIP buildrules supported by the **default** build are:

Table 1. Common Buildrules

Buildrule	Description
vxWorks	ELF file, loadable from the Wind River target loaders
uVxWorks	U-Boot compatible file
vxWorks.bin	Binary file

Determine which buildrules the VIP supports as follows:

```
C:\WindRiver\workspace> vxprj buildrule list myVIP
vxWorks
uVxWorks
vxWorks.bin
```

For more details on buildrules and how to use them, refer to *VxWorks Configuration and Build Guide: Building a VxWorks Kernel Image Project from the Command Line*.

Parent topic: The VxWorks Kernel Image on page 19

4. TARGET BOOT WORKFLOWS

1. Use U-Boot to Boot a uVxWorks Image File With Embedded DTB

The workflow to load VxWorks containing an embedded DTB from U-Boot is similar to the workflow provided in the BSP readme file for your board. There are only two differences:

1. You do not need to load a separate DTB file from the U-Boot shell, as it is contained within uVxWorks.
2. The construction of the bootm command only needs to include the VxWorks load address.

```
=> tftp 0x81000000 uVxWorks
=> bootm 0x81000000
```

2. Use U-Boot to Boot vxWorks.bin

The VxWorks image project (VIP) binary build target vxWorks.bin must be built with an embedded DTB, and will be loaded from U-Boot using a different workflow and a different load address when compared to uVxWorks. Refer to Creating a vxWorks.bin Kernel Image on page 20 for instructions on building vxWorks.bin.

Loading vxWorks.bin from U-Boot on page 23

2.1. Loading vxWorks.bin from U-Boot

Before You Begin

These steps assume:

1. you previously created and built VSB and VIP projects to support embedded DTB and generated the vxWorks.bin build target.
2. you have a TFTP server running and configured to serve files from your VIP default directory.

Procedure

1. Set the target board network parameters from the U-Boot command shell:

```
=> setenv ipaddr 192.168.0.3
=> setenv netmask 255.255.255.0
=> setenv serverip 192.168.0.2
=> setenv bootargs 'cpsw(0,0)host:vxWorks h=192.168.0.2 e=192.168.0.3:ffffff00 u=target pw=vxTa
rget f=0x00'
=> saveenv
```

Note: These boot parameters assume a workstation IP address of 192.168.0.2 and target board IP address of 192.168.0.3. Adjust these IP addresses based on your network configuration.

WNDRVR

Note: If you set bootargs incorrectly, this can cause the target network connection to fail. If this occurs, you must clear bootargs in U-Boot so that VxWorks will default to the bootline specified in the device tree file.

```
=> setenv bootargs
=> saveenv
```

2. Load the VxWorks image file.

```
=> tftp 0x80100000 vxWorks.bin
```

3. Use the U-Boot go command to run the VxWorks kernel image.

```
=> go 0x80100000
```

Parent topic: Use U-Boot to Boot vxWorks.bin on page 23

3. Boot the VxWorks Image Using the Boot Application

The boot application can only boot a traditional VxWorks ELF image configured with an embedded DTB. For further details on creation of this VxWorks image project, refer to Creating a VxWorks Kernel Image with Embedded DTB on page 20.

The boot application assumes that an FTP server is running on the host workstation. Use the following parameters to FTP the VxWorks image from the boot application on the target.

Note: Replace **inet on ethernet (e)**, **host inet (h)**, **user (u)** and **ftp password (pw)** with the settings appropriate for your network and FTP server.

```
boot device          : cpsw
unit number          : 0
processor number     : 0
host name            : host
file name            : vxWorks
inet on thernet (e)  : 192.168.1.100
host inet (h)        : 192.168.1.1
user (u)             : target
ftp password (pw)    : vxTarget
flags (f)            : 0x0
```

4. Summary of Target Boot Workflows

Table 1. VxWorks Boot Workflows

	uVxWorks	**vxWorks.bin**
Boot Loader	U-Boot	U-Boot
DTB Mode	Independent/Embedded	Embedded
Boot Command	bootm	go

5. DEVICE SPECIFIC INFORMATION

1. Network and Ethernet (MAC) Configuration

The target board has a two port network switch interface. Port 0 is the host port, and port 1 is the ethernet port which is controlled by the VxWorks ethernet driver. Port 1 is the device **cpsw0**.

To include the ethernet driver manually, include the component **DRV_END_FDT_TI_DA4CPSW**.

Configuring the Ethernet Address from the U-Boot Console on page 26U-Boot will usually allow the target board ethernet (MAC) address to be updated from the U-Boot serial console.

1.1. Configuring the Ethernet Address from the U-Boot Console

U-Boot will usually allow the target board ethernet (MAC) address to be updated from the U-Boot serial console.

Refer to your target documentation to check if U-Boot supports setting the *ethaddr* variable, and also whether you can save the environment variables from the U-Boot console. If it does then VxWorks will inherit the ethernet address you set from U-Boot, and this new address will persist beyond a reboot. Set the address as follows:

=> **`setenv ethaddr`** *AA:BB:CC:DD:EE:FF* **`;saveenv`**

Parent topic: Network and Ethernet (MAC) Configuration on page 26

2. Timer

Various hardware timers are provided in either the processor core or in the SOC.

DM Timer

Thirty timer modules exist in this device. One is used for the VxWorks system clock, and another for the auxiliary clock. They are supported by adding **DRV_TIMER_FDT_OMAP3** to the VIP.

ARM Generic Timer

This is the ARM Cortex-A7/A8 MPCore generic timer. It is supported by adding **DRV_ARM_GEN_TIMER** to the VIP. It is used as the BSP timestamp driver.

3. I2C

I2C is supported by the BSP. Enable I2C in the VIP by adding the component **DRV_AM38XX_I2C**. The application, BSP or device driver must use the I2C VxBus API to use I2C bus devices. For more details, refer to *VxWorks BSP and Driver Guide: I2C Driver Overview*

I2C EEPROM on page 27
I2C Real Time Clock (RTC) on page 27

WNDRVR

I2C I/O Expander on page 27
I2C I/O Multiplexer on page 27

3.1. I2C EEPROM

EEPROM devices are supported by this BSP. To include EEPROM, add the components **INCLUDE_EEPROMDRV** and **DRV_I2C_EEPROM** to the VIP. Once added, the EEPROM will be used as an I/O device and accessed using standard VxWorks I/O routines open( ), read( ), write( ), ioctl( ), and close( ).

The J721E EVM board has two EEPROM devices. The EEPROM device with address 0x50 in the warkup domain stores board-specific data. The EEPROM device with address 0x50 is on the first controller of the MCU domain and it stores boot information.

Parent topic: I2C on page 26

3.2. I2C Real Time Clock (RTC)

The I2C RTC device on the J721E EVM board is supported by this BSP. Add the component **DRV_I2C_RTC** to the VIP to support it. The RTC device is on the main I2C domain, and has address 0x6f.

Parent topic: I2C on page 26

3.3. I2C I/O Expander

Several I/O expander devices are supported on the J721E EVM board. Refer to the board schematic for connection relationships between the SoC and the TCA6408, TCA6416 and TCA6424 devices. Add support to the VIP with the component **DRV_I2C_GPIO_FDT_MAX_731X**.

Parent topic: I2C on page 26

3.4. I2C I/O Multiplexer

The TCA9543A I2C multiplexer device on the J721E EVM board is supported by this BSP. Add support to the VIP with the component **DRV_I2C_PCA954X**.

Parent topic: I2C on page 26

4. GIC-V3

The ARM generic interrupt controller (GIC) V3 is supported by the BSP. Enable it in the VIP by adding the component **DRV_INTCTLR_FDT_ARM_GIC_V3**.

GIC-V3 ITS on page 27

4.1. GIC-V3 ITS

The ARM GIC V3 interrupt translation services (ITS) is supported by the BSP. Enable it in the VIP by adding the component **DRV_INTCTLR_FDT_ARM_GIC_V3_ITS**.

Parent topic: GIC-V3 on page 27

5. Interrupt Routers

The interrupt router (INTRTR) module provides a mechanism to multiplex M hardware interrupt interrupt sources to N interrupt outputs. All M inputs are selectable to be driven per output N. There is one hardware register per output (MUXCNTL_N) that controls the selection.

Enable it in the VIP by adding the component **DRV_INTR_FDT_TI_SCI**.

6. Interrupt Aggregator

The interrupt aggregator (INTR_AGGR) terminates system events to make them coherently processable by the system hosts. It converts between global events, transmitted on the event transport lane (ETL), and local events that are level sensitive outputs and level or edge sensitive inputs.

Enable it in the VIP by adding the component **DRV_INTA_FDT_TI_SCI**.

7. SPI

The SPI controller driver, called OSPI, is supported by the BSP. Enable it in the VIP by adding three components:

- **DRV_TI_TDA4X_OSPI**
- **INCLUDE_PINMUX_SYS**
- **DRV_PINMUX_FDT_AM3**

OSPI Flash Support on page 28

7.1. OSPI Flash Support

The BSP supports OSPI flash when the component **DRV_XSPI_FLASH** is added to the VIP.

The XSPI flash driver supports the **Micron, mt35xu512aba** flash device, The device information is as follows:

Manufacturer ID:

0x2c

Device ID:

0x5b1a

Extend ID:

0x1041

Page Size:

256 KB

Sector Size:

128 KB

WNDRVR

Chip Size:

64 MB

Parent topic: SPI on page 28

8. Secure Proxy

The Secure Proxy module implements a messaging channel used to communicate between the hardware System Control Interface (TISCI) and Device Management and Security Controller (DMSC).

Enable it in the VIP by adding the component **DRV_FDT_TI_SEC_PROXY**.

9. TISCI Firmware

The Texas Instruments System Control Interface (TISCI) implements the communication protocol between various processing elements in the SoC. It incorporates a set of message formats and a sequence of operations supporting the main goal of TISCI, which is SoC independent communication between elements.

Enable it in the VIP by adding the component **DRV_FIRMWARE_TISCI**.

10. UDMA

UDMA is used for packet oriented communications.

Enable it in the VIP by adding the component **DRV_DMA_FDT_TI_K3_UDMA**.

RINGACC on page 29

10.1. RINGACC

The Ring Accelerator (RINGACC) provides hardware ring acceleration to enable the passing of data between a producer and a consumer. It works as a submodule of UDMA.

Enable it in the VIP by adding the component **DRV_DMA_FDT_TI_K3_RA**.

Parent topic: UDMA on page 29

11. PinMux

The PinCtrl IOMUX module which implements PinMux management is supported in this BSP and must be enabled by default with the inclusion of the component **DRV_PINMUX_FDT_AM3**.

The PinMux module provides several groups of PinMux setting values in the IOMUX device tree nodes, and each group includes the whole PinMux setting of a specified device. The device driver that requires PinMux function should find a supported group handle and write it to its device tree source node.

For example, the wkup_i2c0 configuration references the PinMux group handle in **j721e-evm.dts**:

```
&wkup_i2c0
    {
    status = "okay";
    pinmux-0 = <&wkup_i2c0_pins_default>;
```

The i2c driver must then enable the PinMux setting in the driver attach method. For example:

```
vxbPinMuxEnable (pDev);
```

12. GPIO

The J721E EVM board SoC on-chip general purpose I/O (GPIO) controller is supported and enabled in the VIP by adding the component **DRV_GPIO_FDT_TI_TDA4**. The application, BSP and driver must then use the VxBus GPIO API to use GPIO. For more information, refer to *VxWorks BSP and Driver Guide: GPIO Driver Overview*

13. PCIe

The J721E SoC has four PCIe controllers and each controller supports both Root Complex (RC) and End Point (EP) operation modes. The J721E EVM board has two PCIe x4 lane sockets and one M.2 socket, and they are all supported by this BSP in both RC and EP operation modes. Enable RC mode by including the component **DRV_FDT_TI_TDA4_PCIE_RC** into the VIP. To enable EP mode, include the component **DRV_FDT_TI_TDA4_PCIE_EP** into the VIP. The pcie0 configuration must also be changed to reference EP mode in **j721e-evm.dts**:

```
&pcie0
    {
    pci-mode = <PCI_MODE_EP>;
    num-lanes = <1>;
    status = "okay";
    };
```

Refer to *VxWorks BSP and Driver Guide: Detecting and Configuring Devices with a Device Tree* for more details.

Note: Refer to the *J721E DRA829/TDA4VM/AM752x Processors Silicon Revision 1.0 Technical Reference Manual* to understand how PCIe shares hardware SERDES lanes with USB, CPSW ethernet controller and SGMII. To use PCIe you must make sure CPSW is disabled. For more details, refer to the *TI Processor SDK Linux Automotive Installer package*.

Note: PCIe legacy interrupt mode is unsupported in RC mode because the TI J721e EVM PCIe hardware does not support legacy interrupts in RC mode.

Note: The TI J721e EVM board PCIe in RC mode cannot access the device non-existent function. This is a known issue by TI. It may cause some multi-function PCIe devices to be unsupported on this board.

Note: The PCIe driver in RC mode was tested alongside the Intel Pro/1000 VxBus Enhanced Network driver. The network driver requires the Intel Gigabit CT Desktop Adapter with the WG82574L controller. To include this driver into the VIP, define the component **INCLUDE_GEI825XX_VXB_END**.

WNDRVR

14. Clock

The clock module drives cores and peripherals, and is supported in this BSP. To include the clock module driver in the VxWorks image project (VIP), add the component **DRV_CLOCK_FDT_TI_SCI** to it.

The clock driver is compatible with TISCI firmware version 19.08.00. There are two different clock register methods which depend on whether the property **<ti,scan-clocks-from-dt>** exists in the BSP file **tda4.dtsi**. However, the selected clock register method does not affect how device drivers use clocks.

All of the clocks supported by the BSP are defined in the BSP file **tda4.dtsi**. The name of every registered clock is <&k3_clks_*devid*_*clkid*>. For an explanation of the mapping between peripheral device ids and clock ids, refer to the *J721E DRA829/TDA4VM/AM752x Processors Silicon Revision 1.0 Technical Reference Manual*.

When a driver requires a clock, you must first locate the appropriate clock *phandle* in **tda4.dtsi** and write it to the device tree source node. For example, the *main_uart0* is located in **tda4.dtsi** and assigned to the serial driver:

```
main_uart0:  serial@2800000
    {
    compatible  = "ti,am5-uart";
    reg         = <0x0 0x2800000 0x0 0x1000>;
    interrupts  = <224 0 1 0>;
    interrupt-parent = <&intc>;
    clocks = <&k3_clks 146 0>;
    clock-names = "fclk";
    status = "disabled";
    };
```

When a clock is assigned to a device in **tda4.dtsi**, the VxBus driver must also enable the clock in the driver attach method. This is achieved by the serial driver as follows:

```
pClk = vxbClkGet(pDev, NULL);

if (pClk == NULL)
    return ERROR;

(void)vxbClkEnable (pClk);

xtal = vxbClkRateGet(pClk);
```

15. KDRV

The Texas Instruments remote device framework implements the communication protocol based on the RPMsg between various processors in the SoC. Customers can write KDRV message-based applications to communicate with remote devices.

Customers who need to use KDRV to communicate with remote devices will need to register KDRV based applications in the KDRV framework using the following:

```
LOCAL struct kdrv_driver kdevDriver =
    {
    {NULL, NULL},
    DEVICE_TYPE_ETHSWITCH,
    kdev_switch_init,
    kdev_switch_cb,
    NULL
    };
```

```
kdrv_register_driver(&kdevDriver);
```

Where kdev_switch_init( ) is the KDRV application initialization callback function, and kdev_switch_cb( ) is the data receiving callback function of the KDRV application.

Before sending the request message, we must connect to the remote device:

```
kdrv_connect (kdrvDev);
```

The API provided by KDRV must be called when the request message needs to be sent to the remote device:

```
kdrv_send_request (kdrvDev,  buffer, bufferSize);
```

Enable it in the VIP by adding the component **INCLUDE_RPMSG_KDRV**.

16. Remote Processor

Enable Remote Processor in the VIP

The Cortex-R5 processor is supported by this BSP as a remote processor. It provides essential support for the RPMsg component.

The BSP DTS files must be updated before remote processor support is enabled in the VIP. See Update the BSP DTS File on page 6 for details.

Enable the remote processor in the VIP by adding the component **DRV_FDT_TI_REMOTEPROC_K3_R5**.

There must be a firmware file for the Cortex-R5 processor in the filesystem. The firmware can be stored in ROMFS or MMC.

Store Firmware in ROMFS

The ROMFS filesystem is used by default with a default firmware file path of **/romfs/j7-main-r5f0_0-fw**. The ROMFS filesystem must be enabled in the VIP by adding the component **INCLUDE_ROMFS**.

The firmware file name must not be changed but the firmware file folder path is a VIP parameter with the default value of **"/romfs"**. Change the firmware file folder path with the following command:

```
vxprj parameter setstring VXB_TI_REMOTEPROC_K3_R5_FIRMWARE_PATH "/romfs"
```

Store Firmware in MMC

The firmware file can optionally be loaded from an MMC device instead of ROMFS. It is assumed that the firmware file has been copied to **/mmc0:1/j7-main-r5f0_1-fw** using the following U-Boot command:

```
tftpboot 0x84000000 j7-main-r5f0_0-fw
fatwrite mmc 0:1 0x84000000 j7-main-r5f0_0-fw 0x${filesize}
```

Enable loading the firmware from MMC with the following commands:

WNDRVR

```
vxprj component add DRV_TI_SDHC_CTRL DRV_MMCSTORAGE_CARD INCLUDE_DOSFS_FAT INCLUDE_DOSFS
vxprj component add INCLUDE_FS_DEVICE_MOUNT_VALIDATE
vxprj parameter setstring FS_ORIGIN_DEVICE_NAME "/mmc0:1"
vxprj parameter setstring VXB_TI_REMOTEPROC_K3_R5_FIRMWARE_PATH "/mmc0:1"
```

Note: Only Cortex-R5 **main-r5f0_0** is verified with the default firmware file (**/lib/firmware/j7-main-r5f0_1-fw** on MMC 1 partition 2). The shutdown operation is not supported.

Note: The Cortex-R5 must be loaded and started by U-Boot, before VxWorks is running.

17. Mailbox

The TI OMAP mailbox is supported in this BSP.

The BSP DTS files must be updated before mailbox support is enabled in the VIP. See Update the BSP DTS File on page 6 for details.

Enable mailbox in the VIP by adding the component **DRV_FDT_TI_OMAP_MAILBOX**.

6. KNOWN BSP ISSUES

1. SMP

VxWorks Symmetric Multiprocessing (SMP) is supported when this BSP is used on the supported boards. For details on VxWorks SMP, refer to the *VxWorks 7 Application Programmer's Guide: VxWorks SMP*.

The main issues relating to SMP on this BSP are:

1. MMU
   MMU-less configuration is unsupported.
2. Cache and Bus Snooping
   Cache cannot be disabled in an SMP system. Direct calls to cacheDisable( ) do nothing.
   Only copy-back mode is tested with this BSP. Changing the configuration to write-through mode may result in unpredictable behaviour.
3. CPU Number
   The VIP parameter VX_SMP_NUM_CPUS specifies the number of CPUs assigned to the SMP kernel. The CPU referred to includes threads, hyper-threads and so forth.
   VxWorks supports using less than the total number of CPUs in an SMP system. This permits some CPUs to be used for additional tasks such as dedicated tasks or additional asymmetric operating systems.
   CPUs not assigned to the SMP kernel will remain in the default, power-on reset state.
4. Atomic Operations
   Atomic operations are unsupported in SMP mode on memory regions allocated by a call to cacheDmaMalloc( ).

2. Atomic Operations

Atomic operations, such as the kernel API call vxAtomic32And( ), cannot be used when the CPU data cache is disabled in the VxWorks kernel configuration. This is because the global monitor in the interconnect is unimplemented for the TI J721e EVM development board.

Limitations on atomic operations also apply when the kernel is in SMP mode. Refer to SMP on page 34 for more details.

7. REFERENCES

1. See Also

- J721E DRA829/TDA4VM/AM752x Processors Silicon Revision 1.0 Technical Reference Manual
- TDA4VM Jacinto™ Automotive Processors for ADAS and Autonomous Vehicles Silicon Revision 1.0 S, PRSP36D - FEBRUARY 2019 - REVISED OCTOBER 2019
- PROC078E7(001) SCH.pdf
- PROC079E3(001) SCH.pdf
- TISCI User Guide
- Workbench User's Guide
- VxWorks 7 Platform Programmer's Guide
- VxWorks 7 Application Programmer's Guide
- VxWorks 7 Configuration and Build Guide
- VxWorks 7 ARM Architecture Guide
- VxWorks 7 BSP and Driver Guide